SECURITY AUDIT

Venus

Spoke Pools

FINAL REPORT

Oct 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Venus – Spoke Pools – Audit Report
Website	venus.io
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/VenusProtocol/isolated-pools/tree/5b73f3cbb022a0b34bbf701c017ebac51e4ba8d8/contracts
Resolution 1	https://github.com/VenusProtocol/isolated-pools/tree/fcbecf35c5465b0ae48d534c3421f3f22e4a4fbb

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	7	1		5	1	
Low	16	5	1	10		
Informational	24	5	2	17		
Governance	3	1		2		
Total	50	12	3	34	1	

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

SpokeComptroller

The SpokeComptroller contract is the upgradeable risk and policy controller for one spoke pool. Each VToken calls it before minting, redeeming, borrowing, repaying, liquidating, seizing, and transferring, and the Comptroller updates the corresponding reward indices and evaluates the account's positions across entered markets. PoolRegistry lists markets through supportMarket, while ResilientOracle and DeviationBoundedOracle provide prices for account checks.

The SpokeComptroller is a spoke-specific fork of the shared Comptroller. It owns the spoke rules for supply and liquidation allowlists, per-market liquidation incentives, bounded collateral pricing, and dust account resolution. Market custody remains in VToken; the Comptroller applies market policy and computes account liquidity. Its implementation is deployed behind a spoke-specific beacon, and poolRegistry is immutable.

Appendix: Risk Checks and Price Sources

Borrowing and collateral-factor checks use the bounded collateral and debt prices returned by DeviationBoundedOracle. Borrow and member redeem or transfer paths refresh the protection state before relying on those prices. Liquidation-threshold checks and liquidation seize calculations use spot prices from ResilientOracle, because those values determine liquidation routing and seized amounts.

Appendix: Liquidation and Healing

The Comptroller routes accounts with collateral below minLiquidatableCollateral between two pool-level flows:

- liquidateAccount handles an account whose collateral can cover all debt after each collateral market's liquidation incentive. It executes the supplied liquidation orders and requires all listed borrows to be cleared.
- healAccount handles an insolvent account whose collateral cannot cover all debt. It seizes the account's collateral and repays a blended fraction derived from maxClearableDebt / borrows; the remaining debt is recorded as bad debt by the markets.
- Regular VToken liquidations handle accounts above the pool threshold and enforce the per-market close factor.

Appendix: Market Policy and Permissions

The supply allowlist is configured per market and checks the account receiving vTokens. The liquidation allowlist is pool-wide and checks the liquidator at seizure, with an entry check in healAccount. A market-specific liquidation incentive overrides the pool default. Oracle, risk, pause, cap, allowlist, market listing, and reward distributor configuration are split between Access Control Manager roles, the Comptroller owner, and the configured PoolRegistry.

Privileged Functions

- enterMarketBehalf
- unlistMarket
- setCloseFactor
- setCollateralFactor
- setLiquidationIncentive
- supportMarket
- setMarketBorrowCaps
- setMarketSupplyCaps
- setActionsPaused
- setMinLiquidatableCollateral
- addRewardsDistributor
- setPriceOracle
- setDeviationBoundedOracle
- setMaxLoopsLimit
- setForcedLiquidation
- setMarketLiquidationIncentive
- setSupplyAllowlistEnabled
- setAllowedSupplier
- setLiquidationAllowlistEnabled
- setAllowedLiquidator
- setAccessControlManager
- transferOwnership
- acceptOwnership
- renounceOwnership

Invariants

- A market is listed before account actions that require listed-market status are allowed.
- A market collateral factor is at most 95%, and its liquidation threshold is no lower than its collateral factor and no higher than 100%.
- Borrowing and member redemptions or transfers use bounded collateral and debt prices, while liquidation routing and seizure use spot prices.
- A market with a configured supply allowlist accepts mints only for an allowlisted recipient.
- When the liquidation allowlist is enabled, only an allowlisted liquidator can seize collateral or call healAccount.
- A zero market-specific liquidation incentive resolves to the pool-wide incentive.
- liquidateAccount is used only below the pool collateral threshold when all borrows can be repaid from collateral, while healAccount records any unpaid remainder as bad debt.

Issue_01	Governance controls risk parameters, oracles, market listing and upgrades
Severity	Governance
Description	The spoke pool relies on Venus governance and on the accounts it grants roles through the AccessControlManager. Through ACM-gated functions, governance sets every risk parameter of SpokeComptroller: collateral factors, liquidation thresholds, pool and per-market liquidation incentives, the close factor, minLiquidatableCollateral, supply and borrow caps and action pauses. It also controls the supply and liquidation allowlists, which decide who can supply to each market and who can call liquidateBorrow(), liquidateAccount() and healAccount(), and it can retire a market for good through unlistMarket(). Instant liquidation threshold changes can make positions liquidatable without any notice. setForcedLiquidation() lets liquidators repay any borrow in a market in full, whatever the health of the account and without the close factor. enterMarketBehalf() lets an approved account enter markets on behalf of any user, which changes the collateral backing that user's borrows. The owner can replace the price oracle and the deviation-bounded oracle through setPriceOracle() and setDeviationBoundedOracle(). A wrong or malicious oracle can make every account liquidatable or let accounts borrow against inflated collateral. Replacing deviationBoundedOracle can discard all per-asset protection state, so an asset under active protection silently reverts to spot pricing on both legs, loosening borrow and redeem limits until the new instance is configured. The owner also adds reward distributors and sets maxLoopsLimit. Markets can only be listed through SpokePoolRegistry, which is ACM-gated. The owners of the SpokeComptroller beacon and SpokeVTokenBeacon can replace the implementation of every pool and every market behind them in a single upgradeTo() call.

Recommendations	Keep every role and ownership behind the Normal Timelock or an appropriately secured multisig. Grant enterMarketBehalf() only to audited routers that pass their own caller as the account.
Comments / Resolution	Acknowledged. enterMarketBehalf is now enterMarketForAccount in commit 925c7ad3918fc2e72d2b19e23a850fe4236cf6d9 and will only be granted to the gateway.

Issue_02	Unlisting a market with balances can orphan debt and supplier positions
Severity	**Governance**
Description	unlistMarket requires all actions to be paused, both caps to be zero, and collateral factor to be zero, but it does not require zero borrows or supplier balances. Since REPAY must already be paused before unlisting, borrowers cannot clear debt between the pause and the unlist operation. After isListed becomes false, getAssetsIn filters the market from liquidity and liquidation snapshots. The debt then stops contributing to shortfall and is omitted from healAccount and liquidateAccount. Repayment cannot be restored because preRepayHook rejects unlisted markets and unpausing actions also requires the market to be listed. Relisting is blocked because the market remains in allMarkets and _addMarket rejects duplicates. The unlisted debt can continue accruing while remaining invisible to checks, and suppliers may be unable to redeem or transfer residual positions. A borrower with collateral elsewhere can also borrow without this debt counting toward capacity. Unlisting should therefore preserve a wind-down path or require the market to have no residual positions.
Recommendations	Consider requiring zero borrow and supplier balances before unlisting, and provide a wind-down path that preserves repayment and redemption for residual positions.
Comments / Resolution	Acknowledged.

Issue_03	Unsolicited vToken donations change Hub and YieldGroup accounting
Severity	**Medium**
Description	An allowlisted YieldGroup can receive vTokens through ordinary transfers or mintBehalf: preTransferHook does not restrict transfers into the YieldGroup, and preMintHook checks whether the recipient is authorized rather than whether the payer is the Hub. The adapter values the YieldGroup's entire vToken balance, so these donations increase YieldGroup and Hub NAV without issuing Hub shares to the donor. Donations can also bypass the Hub and YieldGroup deposit queues, pauses, and allocation caps, although the Spoke market's mint pause and supply cap still apply. Direct minting consumes supply-cap headroom and increases balances used in Hub cap calculations, which can affect where later Hub deposits are routed. A donation to an omitted resource can be counted in NAV while remaining unavailable through the active withdrawal queue until access is restored. Additionally, The removeResource function in YieldGroupBase requires a zero VToken balance, preventing resource removal when bad debt leaves unredeemable VTokens or another holder transfers dust VTokens to the YieldGroup. Similarly, removeYieldGroup in HubAdminLib rejects removal when the resource-backed value is nonzero. An attacker can trigger this restriction by donating VTokens worth at least one wei of underlying to the YieldGroup or, where existing VTokens have zero value, donating underlying to the market to raise their value above zero. These strict checks allow unsolicited donations and residual bad-debt positions to obstruct resource and YieldGroup removal.
Recommendations	Consider preventing unsolicited vToken donations and minting. Additionally, consider disallowing removal when the balance exceeds a configurable threshold, rather than requiring it to be zero.
Comments / Resolution	Acknowledged. Venus will ensure that the YieldGroup and the listing seed in the VTreasury hold all liquidity-market vTokens, leaving none available for anyone else to send.

Issue_04	Account health checks use stale interest data
Severity	**Medium**
Description	preLiquidateHook updates prices but does not accrue every entered market before taking the account-wide health snapshot. VToken.liquidateBorrow accrues only the borrowed and seized-collateral markets, while getAccountSnapshot reads stored borrow balances and exchange rates for every entered market. Interest accrued in a third market can therefore be missing from the snapshot. Stale collateral-market interest can understate collateral and allow liquidation of an account that is healthy after accrual; stale debt-market interest can understate debt and prevent liquidation of an account that is actually unhealthy. The same stale values can distort the minLiquidatableCollateral routing check. Refreshing all markets returned by getAssetsIn(borrower) before the snapshot would align the check with the position state used by the batch liquidation paths. Additionally, during VToken transfers, preTransferHook checks the source account's liquidity through _checkRedeemAllowed, which relies on _getHypotheticalLiquiditySnapshot`and assumes interest has already been accrued. Because transfers do not accrue interest first, the check can use stale borrow data and allow a transfer that leaves the account unhealthy after accrual, making it immediately liquidatable and potentially causing bad debt in low-liquidity markets. Similarly, preBorrowHook checks a proposed borrow using stored balances from other entered markets, so stale debt can understate what the account owes and allow excess borrowing. preRedeemHook also uses the same stale account data to check a redemption, so it can approve a collateral release that leaves the account unhealthy once interest is accrued.
Recommendations	Consider accruing all entered markets before health checks in liquidation, borrowing, redemption, and transfer hooks.
Comments / Resolution	Unresolved. Periodic keeper accrual reduces staleness but cannot guarantee that all entered markets are accrued before each account

	health check. Although liquidateAccount refreshes all entered markets, the other affected paths remain unchanged and can still evaluate account health using stale balances and exchange rates. Client acknowledged with the comment “This is inherited from upstream, where only healAccount accrues every entered market before the health check. The spoke adds the same refresh to liquidateAccount and leaves the other checks unchanged. Accruing every entered market would add an accrueInterest() call per entered market to every borrow, liquidation, and redeem, transfer or exit that runs the check. To keep idle markets from going stale, we'll add the spoke markets to the keeper that already calls accrueInterest() periodically on core pool markets.”

Issue_05	Direct suppliers can receive rewards outside Hub share accounting
Severity	**Medium**
Description	SpokeComptroller updates supplier reward indices and accrues rewards to the address holding vTokens in each market. RewardsDistributor.claimRewardToken(holder) transfers the holder’s accrued reward tokens directly to that address. A user who supplies directly to a Spoke market therefore earns and can claim the market’s supplier rewards in their own account. Hub depositors hold their Spoke vTokens through a YieldGroup. The Hub’s share value includes the YieldGroup’s underlying positions and idle underlying, while reward tokens are separate assets and are not included in that accounting. Supplier rewards accrued to the YieldGroup therefore do not increase the Hub share value or get distributed pro rata to Hub depositors. Direct suppliers can collect those rewards while Hub depositors receive only returns reflected in the YieldGroup’s underlying asset value. This creates different reward treatment for equivalent direct and Hub-routed supply.
Recommendations	Consider documenting that rewards are not allocated to hub depositors.
Comments / Resolution	Acknowledged. Venus will not enable rewards for spoke pools.

Issue_06	healAccount bypasses the borrowed market's LIQUIDATE pause
Severity	**Medium**
Description	Regular liquidation and each liquidateAccount order pass through preLiquidateHook, where the borrowed market's Action.LIQUIDATE pause is checked. healAccount follows a separate path: it calls seize and healBorrow directly, without checking that pause. The repayment hook checks Action.REPAY only for a nonzero repayment, so that check does not replace the missing liquidation-pause check in every case. During a LIQUIDATE pause, a caller admitted by the liquidation allowlist can therefore heal an eligible account using the oracle price still available to the Comptroller. The borrower loses its collateral and any unpaid debt is recorded as badDebt in the borrowed market. The affected account must satisfy the normal healing conditions, including the minLiquidatableCollateral threshold.
Recommendations	Consider checking the borrowed market's liquidate pause in healAccount.
Comments / Resolution	Resolved in commit 67933bd272d76eb8d8a09c147e766ebf475d795e

Issue_07	Permissionless healing can shift Hub losses between LPs
Severity	**Medium**
Description	healAccount moves an insolvent borrower's unpaid debt from totalBorrows to badDebt in one transaction, and any caller can choose when that transition occurs while the liquidation allowlist is disabled. The Spoke exchange rate includes badDebt, so this accounting change does not itself reduce the Spoke position's value. A Hub adapter that values the position using cash and totalBorrows while excluding badDebt recognizes the loss only when healing occurs. If an LP exits after the insolvency is visible but before the heal, the exit uses a NAV that still includes the impaired debt; the LPs who remain then absorb the exiting LP's share of the write-off.
Recommendations	Consider including badDebt in Hub valuation.
Comments / Resolution	Acknowledged with the comment “Acknowledged, but we will not include badDebt in Hub valuation. Spoke bad debt cannot be recovered through Shortfall (Issue_18), so a valuation that includes it would never book the loss, and the last LPs to exit would take all of it. Excluding it marks every LP down at the same time. Anyone can heal an eligible account, so the window can be closed as soon as the account qualifies.”

Issue_08	Borrower reward updates can be skipped
Severity	**Medium**
Description	Borrower reward updates can be skipped when a heal's truncated repayment is zero and preRepayHook() is skipped. Batch liquidation can also change a borrower's debt without advancing the borrower-side reward index. As a result, reward accounting may not reflect these borrow-state changes.
Recommendations	Consider advancing the borrower's reward index on these paths if the distributor is expected to account for every borrow-state transition.
Comments / Resolution	Acknowledged.

Issue_09	Liquidations can create bad debt because liquidationThreshold × liquidationIncentive < 1 is not enforced
Severity	**Low**
Description	A liquidation repays R of debt and seizes R × I of collateral, which removes R × I × LT of the account's weighted collateral. The account only becomes healthier if the debt removed is larger than the weighted collateral removed, which requires LT × I < 1. No setter enforces this. setCollateralFactor() sets LT without reading the incentive, and setLiquidationIncentive() and setMarketLiquidationIncentive() set I without reading LT. When LT × I >= 1, every liquidateBorrow() call makes the account less healthy. Liquidators can repeat it until the collateral falls to minLiquidatableCollateral, then call healAccount() to write off the remaining debt. Consider the following → - LT is 0.95, I is 1.1, the close factor is 0.5 and minLiquidatableCollateral is 100 USD. - Alice has 1,000 USD of collateral and 960 USD of debt. She is liquidatable, but her collateral still covers her debt. - A liquidator repays 480 USD and seizes 528 USD. Alice now has 472 USD of collateral and 480 USD of debt, and her shortfall has grown from 10 to 31.6 USD. - Two more calls leave 76 USD of collateral and 120 USD of debt. - healAccount() writes off about 51 USD as bad debt. Venus fixed the same issue in the core protocol as VLC-53 by adding ensureSafeLiquidationParams(), which reverts whenever LT × I >= 1.
Recommendations	Port ensureSafeLiquidationParams() into SpokeComptroller and call it from setCollateralFactor(), setMarketLiquidationIncentive() and setLiquidationIncentive(). The last one must check the new pool default against every market that uses it.

Comments / Resolution	Resolved in commit 82fc9f58434ee8e91c800d564251ec6b8fac993e.

Issue_10	Pool liquidation incentives can fall below the protocol seize share
Severity	**Low**
Description	setLiquidationIncentive enforces a fixed 105% minimum, while markets without an explicit per-market incentive inherit the pool-wide value. The setter does not compare the proposed incentive with each inheriting market's protocolSeizeShareMantissa. If the pool incentive is lowered below 100% plus the protocol share, the protocol's portion can leave less than the repaid value for the liquidator. For example, a 105% incentive with an 8% protocol share gives the liquidator approximately 96.6% of the repaid value in collateral. This can reduce the incentive to liquidate and increase bad-debt risk.
Recommendations	Consider checking the new pool incentive against the protocol seize share of every listed market that inherits the pool value, while retaining the 105% floor for newly listed markets.
Comments / Resolution	Acknowledged.

Issue_11	A fixed trigger threshold leaves smaller oracle deviations unprotected
Severity	**Low**
Description	DeviationBoundedOracle compares the observed spot price with its stored reference in _checkAndTriggerProtection and activates protection only when the configured triggerThreshold is reached. Deviations below that threshold leave protection inactive, so getBoundedPricesView returns spot for both collateral and debt prices. In VToken.sol, borrow calls SpokeComptroller.preBorrowHook. That hook updates resilient-oracle prices and protection state for the borrower's entered markets, then checks the proposed borrow with a collateral-factor snapshot. If the price move remains below the trigger threshold, the snapshot uses spot prices. An increase in a collateral price or decrease in a debt price can therefore raise the borrowing capacity used by the check, allowing a borrow that would exceed capacity at the restored price. VToken.redeem and receipt-token transfers also invoke the Comptroller's redeem or transfer hooks. For a member account, _checkRedeemAllowed updates prices and protection state, then checks the hypothetical position using the same bounded-price path. A sub-threshold move can therefore allow a redemption or transfer that leaves the account less collateralized than it would be at the restored price. The reverse price movement can instead lower capacity and cause otherwise valid borrows, redemptions, or transfers to revert.
Recommendations	Consider setting the threshold with the maximum tolerable borrowing and liquidation error in mind, and validate the resulting exposure against each supported asset's oracle design.
Comments / Resolution	Acknowledged.

Issue_12	Liquidation price observations do not trigger bounded-price protection
Severity	Low
Description	Liquidation flows read prices through _safeGetUnderlyingPrices, but they do not update protection observations in DeviationBoundedOracle. A manipulated price used to value a liquidation can therefore be absent from the window later used by borrowing and collateral checks. For example, if the stored window is $100 to $100 and spot is depressed to $50, a liquidator may repay approximately $22.73 and seize 0.5 collateral tokens at a 1.10 incentive. If spot then returns to $100, unchanged bounds may permit up to $35 of borrowing against those tokens at a 70% collateral factor; recording the $50 observation would instead limit capacity to $17.50, assuming no intervening reset.
Recommendations	Consider updating the protection state for liquidation price observations.
Comments / Resolution	Acknowledged.

Issue_13	Rounding can change liquidation routing at near-equality boundaries
Severity	**Low**
Description	liquidateAccount and healAccount use the same rounded maxClearableDebt but apply different equality conditions. For debt 100e18, collateral 110e18 + 1, and incentive 1.1e18, Spoke calculates maxClearableDebt = 100e18 after division rounds down. At that boundary, healing accepts maxClearableDebt <= borrows and calculates a repayment percentage of exactly 1e18, while batch liquidation rejects because borrows >= maxClearableDebt. With collateral of 110e18 + 2, the rounded value becomes greater than debt, changing the route to liquidation. The original Comptroller's separate coverage calculations can allow both paths for the first value. These examples establish a difference in eligibility, not a material loss. The difference is a few USD mantissa units, not dollars or underlying-token units, and actual execution still depends on repayment and seizure calculations.
Recommendations	Consider aligning the equality conditions used by liquidation and healing, or explicitly document the intended boundary routing.
Comments / Resolution	Acknowledged.

Issue_14	An unheld market's broken feed can block account actions
Severity	**Low**
Description	_checkRedeemAllowed takes a collateral-factor snapshot across every market in accountAssets[redeemer], including entered markets where the account has neither vTokens nor borrows. _safeGetUnderlyingPrices reverts with PriceError if either bounded-price leg is zero. The same gate is used by exitMarket and preTransferHook, so a broken feed for an unheld market can trap membership and prevent redeeming, transferring, or exiting during the outage. Snapshots in preLiquidateHook, healAccount, and liquidateAccount can also fail when the borrower has entered the affected market, blocking liquidation resolution while interest continues to accrue. Skipping markets with both zero vToken balance and zero borrow balance avoids making unrelated positions depend on those feeds.
Recommendations	Consider skipping entered markets where the account has neither a vToken balance nor a borrow balance before requesting prices, including in the snapshot consumers used by liquidation resolution.
Comments / Resolution	Partially resolved in commit 3d5075d5d5dabf1eed53372e873d4ab38ed7db0f. Zero balance markets can now be exited during oracle failures, but redemptions, transfers of nonzero collateral, and liquidations still check unrelated zero-position markets. A broken feed can therefore still block these operations.

Issue_15	mint can accept underlying while minting zero vTokens
Severity	**Low**
Description	Direct calls to VToken.mint() and mintBehalf() can transfer underlying without minting any vToken units. SpokeComptroller.preMintHook() checks the mint pause, market listing, supplier permission, and supply cap, but does not enforce a minimum deposit. After interest accrual, _mintFresh() transfers the underlying and calculates mintTokens = floor(actualMintAmount * 1e18 / exchangeRate). It accepts a zero result, and mintVerify() is a no-op. AdapterSpokeV1.deposit() in already includes a minimum-deposit guard: _oneVTokenUnit(exchangeRateStored()) calculates ceil(exchangeRateStored / 1e18), and an amount below this minimum reverts with DepositBelowOneVToken. This is the underlying amount needed for one raw vToken unit at the stored exchange rate. However, the guard only runs for deposits routed through the adapter; direct calls to either VToken mint entry point do not execute it. For example, with exchangeRate = 2e18, depositing one underlying base unit produces floor(1 * 1e18 / 2e18) = 0 vTokens. The transaction succeeds and the underlying remains in the market, but the recipient receives no receipt tokens for the deposit. mintBehalf() invokes _mintFresh(msg.sender, minter, mintAmount), so the caller pays for the deposit.
Recommendations	Consider requiring a minimum deposit guard in preMintHook. Alternatively, document that direct deposits do not have a minimum deposit guard.
Comments / Resolution	Resolved in commit 6a12108896b839b9f29e16155f5ae6f0332f604a. The preMintHook NatSpec now clarifies that no minimum mint amount is enforced, so a deposit worth less than one vToken unit transfers the underlying but mints no vTokens.

Issue_16	healAccount rounds the repayment down twice, against the protocol
Severity	**Low**
Description	healAccount() rounds the repayment down twice: Exp memory percentage = div_(Exp({ mantissa: snapshot.maxClearableDebt }), Exp({ mantissa: snapshot.borrows })); uint256 repaymentAmount = mul_ScalarTruncate(percentage, borrowBalance); Both steps reduce what the caller repays, and healBorrow() records the difference as bad debt. The first rounding applies to the whole borrow, so the loss grows with the size of the position. The amount per call is small, but it always goes against suppliers.
Recommendations	Round up in both steps, capped so the repayment never exceeds borrowBalance.
Comments / Resolution	Resolved in commit 8d9ae8ba195b8dfa88d69ab60c0138f327f69114.

Issue_17	Borrow cap can be consumed to grief suppliers and borrowers
Severity	**Low**
Description	A malicious user can perform same block borrow-repay (to avoid accruing interest) or supply-redeem to consume supply and borrow caps. Since the preMintHook and preBorrowHook implement the cap checks, they ultimately force revert supplier and borrower transactions.. *if (nextTotalSupply > supplyCap) {* *revert SupplyCapExceeded(vToken, supplyCap);* *}* *if (nextTotalBorrows > borrowCap) {* *revert BorrowCapExceeded(vToken, borrowCap);* *}*
Recommendations	Consider acknowledging the issue and disabling the caps in such cases.
Comments / Resolution	Acknowledged.

Issue_18	Shortfall cannot start bad-debt auctions for spoke pools
Severity	**Low**
Description	Shortfall._startAuction requires the Comptroller to be present in Shortfall's configured poolRegistry. The deployment configures Shortfall with the isolated-pools registry, while spoke pools are registered in SpokePoolRegistry. 028-deploy-spoke-vtokens.ts wires spoke markets to Shortfall but does not make their Comptroller discoverable through that registry. When Shortfall auctions are enabled, startAuction therefore reverts for bad debt in a spoke pool. The corresponding badDebtRecovered path cannot run, leaving bad debt recorded by healAccount unrecovered through Shortfall and its loss with spoke suppliers.
Recommendations	Document Shortfall caveat next to existing ProtocolShareReserve caveat in 024-deploy-spoke-pool-registry.ts.
Comments / Resolution	Resolved in commit 281a68e9a5283b015d8889cd65cd4e49b9050e57. Shortfall caveat was added next to the ProtocolShareReserve note in 024-deploy-spoke-pool-registry.ts

Issue_19	SEIZE-paused collateral remains borrowable
Severity	**Low**
Description	Currently, the account liquidity snapshot continues to count a listed market as collateral even when Action.SEIZE is paused. The pause is enforced later by preSeizeHook, which blocks the seizure itself, but it does not remove the collateral from the borrowing calculation. An account can therefore supply to the paused market, enter it, and borrow from another market against that collateral. If the collateral loses value while the pause remains active, regular liquidation, liquidateAccount, and healAccount all fail when they try to seize it. The debt market remains exposed to the resulting shortfall until the pause is lifted or the collateral is otherwise removed from the account.
Recommendations	Consider excluding seize paused collateral from borrow capacity.
Comments / Resolution	Acknowledged. Venus will set a market's collateral factor to zero when seizures are paused for that market.

Issue_20	liquidateCalculateSeizeTokens floors the seize ratio too early
Severity	Low
Description	liquidateCalculateSeizeTokens calculates a collateral-to-debt ratio and floors that ratio before multiplying it by the actual repayment amount. The final seizure therefore inherits the precision loss of the intermediate ratio, regardless of how large the repayment is. For example, a ratio mantissa of 10.5 is reduced to 10 before scaling, producing a proportional shortfall in the collateral seized; an integer ratio does not incur this particular truncation. For affected borrowed and collateral market pairs, each liquidation underpays the liquidator. If the resulting collateral is worth less than the repayment after the protocol share, liquidations of that pair can become uneconomic, leaving borrowers underwater and bad debt with suppliers of the borrowed market.
Recommendations	Consider preserving precision until the final calculation.
Comments / Resolution	Resolved in commit 7eee90b110d419e4ce5af3469acc77af69b75725.

Issue_21	A zero-cash collateral market makes seizure revert
Severity	**Low**
Description	The seizure flow transfers the protocol's share of collateral in underlying tokens, so it requires available cash in the collateral market. At full utilization, an accrual-time reserve sweep can remove the remaining cash and leave internalCash at zero. A later cross-market seizure then fails when the VToken tries to transfer the protocol share and subtracts it from the empty cash balance. healAccount seizes the borrower's collateral in an all-or-nothing loop, and batch liquidation can still route the account using collateral that the snapshot counts as clearable. Both paths can therefore revert when they reach the zero-cash market; regular cross-market liquidation can fail for the same reason. While cash remains below the required protocol share, accounts backed by that collateral cannot be cleared through these paths and shortfalls can grow into bad debt.
Recommendations	Consider handling protocol-share transfers when collateral cash is unavailable.
Comments / Resolution	Acknowledged.

Issue_22	Prior liquidations can bypass the close-factor limit
Severity	**Low**
Description	The Comptroller compares minLiquidatableCollateral with totalCollateral, which is accumulated at spot prices without liquidation-threshold weighting. Regular liquidation reduces this value as collateral is seized, so a liquidator can perform a series of individually close-factor-bounded liquidations and bring an account that began above the threshold below it. Once the threshold is crossed, the account becomes eligible for liquidateAccount or healAccount, whose whole-account flows are not limited by the regular close factor. The liquidator can then close the remaining debt in a batch or seize the remaining collateral through healing. This allows a liquidator to reach a full-position outcome through multiple calls even though each regular liquidation separately respects the close factor.
Recommendations	Consider using a threshold that cannot be crossed through prior liquidations.
Comments / Resolution	Acknowledged.

Issue_23	Supply allowlist can be bypassed through transfers and liquidations
Severity	**Low**
Description	When isSupplyAllowlistEnabled is active, preTransferHook does not check whether the VToken transfer's source and recipient satisfy isAllowedSupplied, so an approved supplier can transfer VTokens to an unapproved address. The preSeizeHook also does not check the liquidator, allowing an unapproved liquidator to receive seized VTokens from an allowlisted market and transfer them onward. Enabling the liquidation allowlist would also restrict permissionless liquidations.
Recommendations	Consider enforcing the supply allowlist during transfers and liquidations.
Comments / Resolution	Acknowledged.

Issue_24	Malicious users can perform same token leverage looping to extract rewards
Severity	**Low**
Description	Rewards from the reward distributors are meant to attract liquidity providers/suppliers and borrowers in low-liquidity markets. This is because these markets offer a higher percentage of rewards, assuming similar or faster supply speeds among multiple other high-liquidity markets. The issue is that anyone can game this to extract rewards from the reward distributors by performing same-token leveraging. For example, the malicious user can supply to the USDC market, borrow USDC from it, re-supply, borrow and so-on in a repetitive manner. Since both the supply-side and borrow-side earn rewards based on VToken balance and debt borrowed respectively, this allows existing market liquidity to be recycled to gain higher rewards. For this to be profitable, the notional value of the rewards need to exceed the interest paid. This is dependent on multiple factors such as notional value of reward token, debt token, supply speeds in reward distributors, interest rates etc. In markets where suppliers are less, whales could capitalize on this by earning back most or a large portion of the interest. However even when this is not fully profitable, this could fall under a griefing category where rewards are intentionally misallocated.
Recommendations	Consider acknowledging this issue or explicitly documenting intended economics surrounding rewards.
Comments / Resolution	Acknowledged. Venus will ensure that no market simultaneously allows supply, use as collateral, and borrowing.

Issue_25	One failed accrual blocks batch liquidation and healing
Severity	**Informational**
Description	Before taking its account snapshot, _refreshMarkets calls accrueInterest() on every market returned by getAssetsIn. Both liquidateAccount and healAccount use this refresh, and the loop does not isolate a failing market or skip memberships with zero balances. If one listed market persistently reverts during accrual, its failure reverts the entire liquidation or heal for every account that has entered it. Market membership does not require a position, so a borrower can enter a failing market with no balance and still make that market part of the refresh. For an under-threshold account, regular liquidation is unavailable, leaving both whole-account routes blocked while the accrual failure continues. Any resulting bad-debt growth depends on a listed market having a persistent accrual failure under its deployed rate model, reserve configuration, or token behavior.
Recommendations	Consider preventing one failed accrual from blocking unrelated liquidation routes.
Comments / Resolution	Acknowledged.

Issue_26	Liquidity preview omits borrows in unentered markets
Severity	**Informational**
Description	getHypotheticalAccountLiquidity applies a hypothetical borrow or redemption while iterating the account's entered markets. If vTokenModify is not among those markets, the view never applies the hypothetical borrow amount and reports more available liquidity than the account would have after borrowing. The state-changing borrow path behaves differently: preBorrowHook first adds a new borrower to the market, then takes the liquidity snapshot and includes the borrow. As a result, the view can mislead a client or frontend and cause it to submit a borrow that reverts for insufficient liquidity, but the inaccurate preview does not bypass the on-chain borrow check.
Recommendations	Consider including unentered markets in borrow previews.
Comments / Resolution	Acknowledged. The issue will be resolved in a deferred pull that is outside of the review.

Issue_27	Dust collateral can block healing on zero-value transfers
Severity	**Informational**
Description	healAccount attempts to seize every nonzero collateral balance in its all-or-nothing loop. A third party can transfer dust vTokens to a borrower, and the seizure calculation can round the protocol's share of that balance down to zero. The VToken still attempts to transfer that zero amount of underlying to the protocol reserve; an underlying token that rejects zero-value transfers makes the seizure, and therefore the whole heal, revert. This condition depends on the collateral's exchange rate, liquidation incentive, protocol share, and the underlying token's zero-transfer behavior. It delays bad-debt recognition for the affected account rather than taking funds from the borrower, and a healer can clear the condition by adding enough collateral before the seizure.
Recommendations	Consider skipping zero-value transfers.
Comments / Resolution	Acknowledged.

Issue_28	Deregistering a YieldGroup can strand accrued Spoke rewards
Severity	**Informational**
Description	Spoke supplier rewards accrue to the vToken holder, which is the YieldGroup for Hub-routed positions. RewardsDistributor.claimRewardToken(holder) transfers the accrued reward tokens to that holder. YieldGroupBase.totalAssets() counts underlying positions and idle underlying, so reward tokens held by the YieldGroup do not contribute to Hub share value or get distributed to Hub depositors. Hub.removeYieldGroup() removes a YieldGroup from the Hub registry and both routing queues without transferring unrelated reward-token balances. Any Spoke rewards accrued to the removed group remain at that contract address, outside active Hub accounting and routing, and are not automatically carried into another YieldGroup. Hub shareholders therefore do not receive those rewards through their share value.
Recommendations	Consider claiming or sweeping accrued Spoke reward tokens before removing a YieldGroup.
Comments / Resolution	Acknowledged.

Issue_29	Spot prices remain in use when bounded pricing is disabled
Severity	**Informational**
Description	SpokeComptroller routes several price-dependent operations through DeviationBoundedOracle, but bounded pricing is not enabled for every underlying. When bounded pricing is disabled for an asset, these paths can still use the spot price returned from the resilient oracle. This matters most for collateral or debt assets whose resilient-oracle configuration relies on a single main oracle. In that configuration, the bounded layer does not provide an independent source or range to limit a manipulated spot observation, so the relevant Comptroller checks retain the spot-price exposure. The impact depends on which flows use the asset and how its resilient-oracle configuration is composed. Verify that bounded pricing is enabled for every supported collateral and debt asset, or that the spot-price behavior is intentional for that asset.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_30	Borrow cap can be bypassed due to continuous interest accrual
Severity	**Informational**
Description	The preBorrowHook function enforces the borrow cap by comparing totalBorrows + borrowAmount against the configured borrowCap. However, this restriction is only enforced when a new borrow is initiated. Interest continues to accumulate on outstanding debt, increasing totalBorrows whenever accrueInterest() is executed, without checking the configured borrow cap. Consequently, a market operating close to its borrow cap can exceed that limit solely through interest accrual, even when no additional borrowing occurs. For example, if the borrow cap is 1,000,000 tokens and the outstanding debt is 999,000 tokens, accruing another 2,000 tokens in interest increases the total debt to 1,001,000 tokens, exceeding the configured cap. Although further borrowing is restricted until the debt falls below the cap, the borrow cap does not represent a strict upper bound on total outstanding debt. This is an expected characteristic of interest-bearing lending markets and should be considered when configuring risk limits..
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_31	Supply cap understated incorrect mintAmount comparison
Severity	**Informational**
Description	The supply cap is checked against the requested mintAmount passed as a parameter, not the received amount. The preMintHook uses mintAmount, while Vtoken._mintFresh later mints against actualMintAmount = _doTransferIn(payer, mintAmount), which is smaller for fee-on-transfer underlying. The direction is conservative and the cap under-counts. For any underlying asset where received can exceed requested, such as, a rebasing token that rebases on transfer or transfer-reward tokens - the cap is evaluated incorrectly.
Recommendations	Consider checking the supply cap in the mintVerify hook function instead.
Comments / Resolution	Acknowledged.

Issue_32	Public hook functions can be directly called
Severity	Informational
Description	All hook functions in the SpokeComptroller are public and can be called directly to execute on stale accrual and exchange rates e.g. updating borrow reward token indexes on stale market borrow index. Direct calls also allow spamming RewardsDistributor event emissions. For example, calling preMintHook calls the RD supply reward index update functions, where RewardTokenSupplyIndexUpdated and DistributedSupplierRewardToken both fire on the no-op call. An offchain reader can be misled by this. This also makes the onlyComptroller check on the RD redundant as well.
Recommendations	Consider acknowledging this or restricting the hook functions only to VTokens. This would also avoid any future risk in case of upgrades.
Comments / Resolution	Acknowledged. Venus will not enable rewards for spoke pools.

Issue_33	Incorrect MAX_COLLATERAL_FACTOR_MANTISSA comment in setCollateralFactor
Severity	Informational
Description	Function setCollateralFactor mentions that the collateral factor should be less than or equal to 0.9. However the MAX_COLLATERAL_FACTOR_MANTISSA currently stores a value of 0.95 instead. *// Check collateral factor <= 0.9* *if (newCollateralFactorMantissa > MAX_COLLATERAL_FACTOR_MANTISSA) {* *revert InvalidCollateralFactor();* *}*
Recommendations	Consider updating the comment or the MAX_COLLATERAL_FACTOR_MANTISSA value hardcoded.
Comments / Resolution	Resolved in commit 06657dbf71921aaa472494b422e9347fd7c5d109.

Issue_34	Reward distributor cannot be removed
Severity	Informational
Description	Reward distributors cannot be removed from the SpokeComptroller storage. An RD can become unused if a reward token is not as appealing to a user or if it is retired through grantRewardToken functionality or setting speeds to 0 to stop accrual. However it still remains in the comptroller and every hook pays for it. This also consumes an extra slot in the max loops limit.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_35	Additional pool registries are not recognized by RiskFundConverter
Severity	**Informational**
Description	ProtocolShareReserve supports additional pool registries, but RiskFundConverter still queries only its own poolRegistry. If a Spoke exists only in a newly added PSR registry and PSR allocates a nonzero share to the converter, PSR.releaseFunds(spoke, [asset]) transfers the allocation and calls RiskFundConverter.updateAssetsState. ensureAssetListed then queries the converter's original registry, receives address(0), and reverts with MarketNotExistInPool. PSR _releaseFund does not catch that revert, so the affected release transaction rolls back, including destination transfers, and the funds remain in PSR. Distribution is a separate transaction, so this does not directly block liquidations. Supporting additional registries only in the deposit membership check would still leave their pools out of conversion accounting. Both the membership check and getPools() need to merge results from all configured registries with deduplication.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged.

Issue_36	Lowered collateral incentives affect healing repayment and batch-liquidation eligibility
Severity	Informational
Description	healAccount and liquidateAccount both use maxClearableDebt, which discounts each collateral market by its liquidation incentive. Lowering an individual collateral incentive increases this value because each unit of collateral can cover more debt when the liquidator receives a smaller incentive amount. In healAccount, the increased value can raise the repayment percentage applied across the borrower's debts, while the liquidator still seizes all remaining collateral. In liquidateAccount, it can make more accounts pass the collateral-coverage check. The orders must still repay every borrow, but the lower incentive yields fewer collateral tokens for each unit repaid. For example, if an incentive falls from 110% to 105%, the discounted collateral value rises, potentially moving an account across the batch-liquidation eligibility boundary. The two functions therefore respond differently: healing changes the repayment fraction, while batch liquidation eligibility expands as its collateral reward falls.
Recommendations	Consider requiring the whitelisted liquidator wrapper to enforce slippage limits when it executes either liquidation path. If direct liquidations are to be supported, consider including slippage for collateral to be received by liquidator
Comments / Resolution	Acknowledged. Lowering incentives will be a governance decision that moves value from liquidators to borrowers.

Issue_37	updatePrices does not refresh bounded-price state
Severity	**Informational**
Description	updatePrices updates prices in the configured ResilientOracle. It does not update the observations or protection state maintained by DeviationBoundedOracle. As a result, an operator or integration that calls only updatePrices can leave bounded prices stale relative to the refreshed resilient-oracle values. Flows that consult the bounded oracle continue to use its prior window until the bounded-price update path is called separately.
Recommendations	Consider updating both oracle layers through a single operational path, or document that bounded-price updates must be performed separately.
Comments / Resolution	Resolved and documented in commit 7bb4dc6715a74d60b937e2acfc35c79323ea00b7.

Issue_38	maxClearableDebt truncation introduces boundary precision differences
Severity	**Informational**
Description	maxClearableDebt calculates each discounted collateral value as (collateralValue * 1e18) / liquidationIncentiveMantissa. Scaling before division preserves fixed-point precision, and each per-market division loses less than one USD mantissa unit, excluding earlier valuation rounding. The rounded value can still affect a strict eligibility boundary. With collateral 110e18 + 1, debt 100e18, and incentive 1.1e18, the result rounds to 100e18; liquidateAccount then rejects equality, while healAccount accepts it and calculates a full repayment percentage. This example routes to full repayment and collateral seizure with only dust extra collateral, rather than creating bad debt by itself. When maxClearableDebt < borrows, truncation can also slightly reduce the healing repayment percentage. Conversion into underlying token amounts adds further rounding that is not bounded by the USD division error, so supported low-decimal or unusually priced assets should be checked separately.
Recommendations	Consider acknowledging the issue.
Comments / Resolution	Acknowledged

Issue_39	The liquidation allowlist is global across vTokens
Severity	**Informational**
Description	setLiquidationAllowlistEnabled enables authorization at the Comptroller level rather than separately for each vToken. Once enabled, an approved liquidator can act on any eligible borrow position in the pool and seize eligible collateral from any vToken market. This is appropriate only if the intended policy grants pool-wide authority. If some markets require a different set of liquidators, the global allowlist grants each approved address broader reach than that market-specific policy would permit.
Recommendations	Consider documenting whether pool-wide authorization is intended
Comments / Resolution	Acknowledged.

Issue_40	Paused repay or seize actions block healAccount
Severity	**Informational**
Description	healAccount processes all collateral and borrow markets in one transaction. Each healBorrow re-enters preRepayHook, and each collateral seizure re-enters preSeizeHook; those hooks revert when the corresponding market's REPAY or SEIZE action is paused. Because the operations are atomic, a pause on any involved market reverts the whole heal, including legs whose markets are not paused. This can prevent bad-debt recording for the account even when the other legs could otherwise be processed.
Recommendations	Consider allowing healing to skip or defer paused-market legs, or document that healing requires every involved repay and seize action to be unpaused.
Comments / Resolution	Resolved and documented in commit 610faaf8d572a2791500e26da56adf6c86927d10.

Issue_41	Zero clearable debt can let healAccount record debt without repayment
Severity	**Informational**
Description	healAccount prices only listed markets returned by getAssetsIn(user). If listed collateral is zero while listed borrows remain, maxClearableDebt is zero. The coverage check rejects only when it is greater than borrows, so the calculated repayment percentage becomes zero instead of reverting. The loop can then call VToken.healBorrow(..., 0). That path skips transfer-in, records the remaining principal as bad debt, and clears the borrow without any tokens moving. This state can arise after regular liquidation removes collateral or when collateral remains in an unlisted market; in the latter case the borrower may retain a frozen collateral claim that could become available if the market is relisted. The optional liquidator allowlist does not stop this when disabled, as it is by default. A positive but sufficiently small maxClearableDebt / borrows can also truncate to a zero percentage. Rejecting a zero repayment percentage when debt remains prevents this path independently of allowlist configuration.
Recommendations	Consider reverting when outstanding borrows produce a zero repayment percentage, and include or explicitly reject positions whose collateral is in unlisted markets.
Comments / Resolution	Acknowledged.

Issue_42	liquidateAccount can omit debt added during a callback
Severity	Informational
Description	liquidateAccount caches getAssetsIn(borrower) before executing liquidation orders and uses the same array for its final zero-debt checks. Under a reachable callback path, _seize can transfer a protocol underlying share after updating collateral state, and the callback can call borrowBehalf in a newly entered market through an approved delegate. preBorrowHook adds that market to borrower membership and checks liquidity against the post-seizure collateral. The final liquidation loop still uses the earlier market list, however, and can return with the new debt outstanding. The callback path requires a supported dependency that can make the call; an arbitrary malicious token is not sufficient. The new borrow also has to pass liquidity checks, so this does not alone establish bad debt or theft. Fetching getAssetsIn(borrower) again before the final debt checks includes markets entered during order execution.
Recommendations	Consider fetching getAssetsIn(borrower) again immediately before final debt checks, while retaining the original list for market refreshing.
Comments / Resolution	Resolved in commit a14d365a898b1700cb670bca1fecb2f1a019a2ca

Issue_43	Debt value truncates toward zero, so dust debt is invisible to the solvency check
Severity	Informational
Description	_accumulateMarketPosition() adds each market's debt with mul_ScalarTruncateAddUInt(), which rounds down. A position where debtPrice × borrowBalance is below 1e18 adds nothing to the measured debt. An account that owes only such dust therefore shows no debt at all. Its shortfall is zero, it cannot be liquidated, and healAccount() reverts with InsufficientShortfall. Collateral also rounds down, which favours the protocol, but debt rounds in the borrower's favour.
Recommendations	Round the debt term up in _accumulateMarketPosition() so measured debt never understates what is owed.
Comments / Resolution	Acknowledged.

SpokeComptrollerInterface

The SpokeComptrollerInterface declares the shared Comptroller hooks together with the events and custom errors exposed by the spoke implementation. It lets VToken contracts, registries, integrations, and off-chain consumers compile against the Comptroller's observable behavior without importing its implementation.

The spoke-specific public storage getters are declared separately in SpokeComptrollerViewInterface. The implementation inherits SpokeComptrollerStorage as a sibling base, so its generated public getters cannot satisfy functions declared on this interface base. The separation preserves the implementation's inheritance and storage layout.

Appendix: Observable Comptroller Surface

The interface includes callback hooks for market actions, pool-level liquidation and healing calls, market and account views, and events and errors used to describe policy failures. SpokeComptrollerViewInterface exposes the spoke-only getters for allowlists, liquidation incentives, pause state, and the bounded oracle.

SpokeComptrollerStorage

The SpokeComptrollerStorage contract defines the state layout inherited by SpokeComptroller. It stores market risk parameters, account membership, caps, pauses, reward distributors, allowlist configuration, liquidation incentives, and the bounded oracle address. It contains no external management logic; the implementation reads and updates these fields.

This storage base is maintained as a spoke-specific fork of ComptrollerStorage, so spoke-only state can evolve without changing the shared implementation used by other pools. It retains a 42-slot gap to preserve the reserved storage footprint, but its layout is not interchangeable with the shared Comptroller layout. Spoke pools upgrade through their own beacon.

Appendix: Account Risk State

Market records listing status, collateral factor, liquidation threshold, and account membership. AccountLiquiditySnapshot carries the aggregate collateral, debt, effects, liquidity, shortfall, and liquidation-only maxClearableDebt. The latter is meaningful only for snapshots weighted by liquidation thresholds. LiquidationOrder and RewardSpeeds hold data passed through batch liquidation and reward views.

Appendix: Spoke-Specific Configuration

Supply allowlists are per market. The liquidation allowlist is pool-wide. liquidationIncentives overrides _poolLiquidationIncentiveMantissa per collateral market, and deviationBoundedOracle is used by collateral-factor risk checks.

SpokePoolLens

The SpokePoolLens contract is a stateless read helper for spoke pools. It combines shared pool and market data with spoke-only settings so clients can query a pool through one lens, including allowlists, liquidation thresholds and incentives, bounded pricing configuration, bad debt, and reward information. It complements PoolLens, which continues to read pools in the shared registry.

The lens reads from PoolRegistry, SpokeComptroller, VToken, ResilientOracle, DeviationBoundedOracle, and RewardsDistributor. The registry address is supplied to registry-wide view functions, so one deployment can serve multiple spoke pools. Balance helpers that call current balance functions may update market interest state; the lens itself stores no state and is versioned by redeployment.

Appendix: Spoke Pool and Market Data

getSpokePoolData and related functions return pool metadata plus market rates, balances, caps, pause flags, collateral factors, liquidation thresholds, effective incentives, allowlist state, forced-liquidation state, and bounded-pricing status. Permission helpers expose whether supply or liquidation restrictions apply to an account.

Appendix: Price and Reward Views

vTokenUnderlyingPrice reports the spot price from ResilientOracle. The Comptroller may use bounded collateral and debt prices for borrowing and member redemption checks, so this lens field is not itself the bounded risk price. Reward views combine distributor indices and speeds with account balances to report accrued and not-yet-distributed rewards.

Issue_44	Bounded prices are not exposed by SpokePoolLens.vTokenUnderlyingPrice
Severity	**Informational**
Description	SpokePoolLens.vTokenUnderlyingPrice() returns the spot price from comptroller.oracle().getUnderlyingPrice(vToken). Borrowing-capacity and collateral-factor checks instead use bounded collateral and debt prices from _safeGetUnderlyingPrices, while liquidation-threshold checks use spot for both legs. During protection, collateral is valued at the lower of spot and the window minimum, and debt at the higher of spot and the window maximum. For 100 supplied tokens at an 80% collateral factor, a $120 spot price implies $9,600 of capacity in a client calculation, while a $100 bounded collateral price yields $8,000 on-chain. The lens is still correct as a spot-price view, and the discrepancy cannot bypass on-chain bounds. A frontend using it for risk calculations can offer borrows or withdrawals that revert, or understate debt when the bounded debt price is higher. Exposing bounded collateral and debt prices separately lets clients display the values used by the corresponding checks.
Recommendations	Consider preserving the spot underlyingPrice field and exposing bounded collateral and debt prices from getBoundedPricesView separately.
Comments / Resolution	Partially resolved in commit c676d94583335302aa6b0aa14ddefc0107ba82b4. In case getBoundedPricesView() reverts for a market, spokeVTokenPrices() also reverts, even when a spot price is available.

Issue_45	spokeVTokenMetadata in SpokePoolLens returns incorrect exchange rate
Severity	Informational
Description	Function spokeVTokenMetadata in SpokePoolLens returns incorrect exchange rate since it retrieves the pre-accrual stored exchange rate.
Recommendations	Consider performing a virtual accrual before returning the exchange rate. Alternatively, acknowledge the issue.
Comments / Resolution	Partially resolved in commit c676d94583335302aa6b0aa14ddefc0107ba82b4. _accrued() does not enforce VToken's maximum borrow-rate check, so the Lens may return projected exchange rates even when exchangeRateCurrent() would revert.

Deployments

The deployment scripts establish the spoke pool's separate registry, its Comptroller proxy and dedicated beacon, the spoke VToken implementation and beacon, and the pool data lenses. They use network configuration for governance addresses and block or timestamp based market parameters. The Comptroller implementation and VToken markets use dedicated beacons so spoke upgrades remain separate from the shared isolated-pools deployments.

Appendix: Spoke Pool Registry

024-deploy-spoke-pool-registry deploys PoolRegistry as SpokePoolRegistry behind an OptimizedTransparentUpgradeableProxy, initializes it with the configured Access Control Manager, verifies the proxy wiring, and nominates the Normal Timelock as owner. It reuses the configured DefaultProxyAdmin when available. Registry and Comptroller role grants and pool configuration are handled in the governance listing flow.

Appendix: Spoke Comptroller

025-deploy-spoke-comptroller deploys SpokeComptrollerImpl with the dedicated registry as an immutable, creates a spoke-specific UpgradeableBeacon, and deploys the Comptroller beacon proxy initialized with the configured loop limit and Access Control Manager. It verifies the registry, manager, beacon implementation, and loop limit before nominating the Normal Timelock as owner. Comptroller ownership is two-step, so the nominated owner must accept it before owner-gated configuration is available.

Appendix: Spoke VToken Beacon

027-deploy-spoke-vtoken-beacon deploys the spoke VToken implementation with the network time model, annualization value, and maximum borrow rate, then deploys SpokeVTokenBeacon and transfers beacon ownership to the Normal Timelock. Market proxies are deployed and registered later through pool listing. This beacon remains separate from the shared VTokenBeacon.

Appendix: Pool Lenses

026-deploy-spoke-pool-lens deploys and verifies the stateless SpokePoolLens using the network's block or timestamp based parameters. 007-deploy-pool-lens.ts deploys the shared PoolLens with the same network-specific timing configuration. The two lens deployments serve their respective spoke and shared pool registries.

Issue_46	Incomplete verification in 025-deploy-spoke-comptroller
Severity	Informational
Description	The 025-deploy-spoke-comptroller script verifies upgradeable beacon => implementation, comptroller => pool registry, comptroller => ACM, and the max loops limit. It never verifies proxy => beacon, which is the link that decides which implementation the live pool actually follows and which beacon a future upgradeTo VIP has to target.
Recommendations	Consider acknowledging the issue or performing the verification.
Comments / Resolution	Resolved in commit a01125482feef2dc62ac1e04630e31f4e593332b.

Issue_47	Deployment can activate markets before required Comptroller configuration
Severity	Informational
Description	Deployment 025 does not configure the deviation-bounded oracle, supply allowlist, liquidation allowlist, or ownership acceptance as part of the deployment script. A fresh Comptroller has no listed markets, so these omissions alone do not expose an active pool; the risk arises if markets are listed before setup is complete. The deviation-bounded oracle defaults to zero, which can make borrowing and collateral-member redemption or transfer liquidity checks fail while deposits are still accepted. The supply allowlist defaults disabled, so adding allowed suppliers without enabling the restriction does not limit minting. The liquidation allowlist also defaults disabled. Enabling it without first approving liquidators and heal keepers blocks liquidation and healing. setPriceOracle also defaults to zero, but the normal PoolRegistry.addPool flow rejects that state. acceptOwnership is needed to transfer owner control to the nominated timelock. Until accepted, the deployer retains owner privileges and timelock owner-only calls fail. The registry flow already sets the close factor, pool incentive, and minimum liquidatable collateral during addPool, then lists markets and sets their factors and caps in addMarket. Configure the remaining required controls before markets become active, or keep their actions paused until setup is complete.
Recommendations	Consider completing oracle and allowlist configuration and accepting ownership before enabling market actions, or keeping markets paused until that setup is complete.
Comments / Resolution	Acknowledged.

Issue_48	Incomplete redeploy in 025-deploy-spoke-comptroller can overwrite SpokeComptrollerImpl.json
Severity	Informational
Description	A redeploy in the 025-deploy-spoke-comptroller script overwrites deployments/<network>/SpokeComptrollerImpl.json on any change in the SpokeComptroller. However even though the implementation deployment succeeded, the script would throw an error since the beacon (skips new deployments) would still point to the old implementation since it needs to use upgradeTo.
Recommendations	Consider acknowledging the issue and similar to the 026 script, archive it as SpokeComptrollerImplR<n>.json first, next to the source.
Comments / Resolution	Acknowledged.

Additional Issues Outside of Scope

Please note that the audit did not cover the issues within this section and hence any changes made here, will require additional resources outside of the resolution round.

Resolution

Please note that this was a limited courtesy review of the changes, not a full resolution round or security audit. It does not confirm that all reported issues have been identified. The code should not be assumed to be secure based on this review. A full audit is required.

Issue_49	Unlisted comptroller markets can be registered
Severity	**Governance**
Description	Function validateRegistration allows adding unlisted comptroller markets to the YieldGroup. This causes all supply and withdrawals to be blocked, causing a revert in the upstream Hub reallocation operation. *function validateRegistration(address resource) external view override {* *ISpokeComptroller comptrollerContract =* *ISpokeComptroller(IVTokenIsolated(resource).comptroller());* *if (* *comptrollerContract.isSupplyAllowlistEnabled(resource) &&* *!comptrollerContract.isAllowedSupplier(resource, msg.sender)* *) {* *revert SupplyNotAllowed(resource, msg.sender);* *}* *}*
Recommendations	Consider implementing a check to ensure the market is listed.
Comments / Resolution	Fixed (unverified). The fix was implemented in the private venus-liquidity-hub repository at commit: c1010a7f304b30c1230247cc00ac2fc665e56b52. As the repository was inaccessible during our fix review, we could not independently verify the production changes. The fix is therefore marked as implemented based on the team's confirmation, but remains unverified by us.

Issue_50	Differing adapter and market pricing under-reports available liquidity leading to the YieldGroup/Hub to incur loss
Severity	Medium
Description	The AdapterSpokeV1 contract calculates the total underlying assets using function _recoverableValue, which excludes bad debt from the accounting. This causes the market to price redeemable VTokens differently from the adapter's pricing. However by doing so, the YieldGroup under-reports the available liquidity at that specific point in time and reports liquidity as if it is the last supplier to exit (the one to incur the entire loss), which is incorrect since the VToken market is first-come first-served. *function _recoverableValue(address resource, uint256 vBal) private view returns (uint256 value) {* *uint256 supply = IVTokenIsolated(resource).totalSupply();* *// Unreachable while `vBal` is non-zero, which every caller has already established. Kept as* *// a division guard rather than an assumption about a contract this one does not own.* *if (supply == 0) return 0;* *uint256 backing = IVTokenIsolated(resource).getCash() + IVTokenIsolated(resource).totalBorrows();* *uint256 reserves = IVTokenIsolated(resource).totalReserves();* *// The market keeps `cash >= totalReserves` (both the redeem and borrow paths check cash net* *// of reserves), so this only guards a pathological state rather than an expected one.* *if (backing <= reserves) return 0;* *return (vBal * (backing - reserves)) / supply;* *}*
Recommendations	Consider acknowledging and documenting the following points: - _recoverableValue is a pro-rata estimate after bad debt, not a

	recoverable amount - maxWithdraw intentionally under-reports - repeated withdrawals converge to the market-rate value. Alternatively fix the pricing by following the market pricing.
Comments / Resolution	Acknowledged. Venus will keep the current pricing because the adapter deducts only the Hub's share of bad debt. A withdrawal race would require another holder to exit first. Venus will ensure that only the YieldGroup can supply, with the governance-controlled VTreasury seed as the only other position. Any loss shifted to the Hub is therefore limited to the seed's share of bad debt.